

Pitanja sa ispita i kolokvijuma

I 6.12.2007. i 26.12.2008.

II 19.1.2008. i 21.1.2009. i 21.1.2011.

jun 2006. - novembar 2007. - decembar 2007. – januar 2008. – mart 2009. – april 2010.

1. Navesti mere performansi paralelnih sistema i njihove definicije. x 6

Odg!ppt1 Ubrzanje predstavlja ocenu o dobijenom poboljšanju sa stanovišta performansi. Ovo ubrzanje se može izraziti kao vreme izvršenja programa na arhitekturi bez poboljšanja podeljeno sa vremenom izvršenja programa na arh. sa izvedenim poboljšanjem. Za paralelni sistem, može se izraziti kao $S(n)=T(1)/T(n)$, pri čemu je $T(1)$ vreme izvršenja programa na jednoprocesorskom sistemu, a $T(n)$ vreme izvršenja programa na n -procesorskom sistemu.

Efikasnost

Definiše se kao odnos ubrzanja i broja procesora. $E(n)=S(n)/n$, $0 \leq E \leq 1$. Daje ocenu o srednjoj iskorišćenosti procesora u sistemu kada su svi procesori angažovani na rešavanju istog problema. Ako se zanemare U/I aktivnosti, efikasnost jednoprocesorskog sistema je 1 (100%). Ubrzanje i efikasnost ne treba posmatrati odvojeno jer sa porastom broja procesora ubrzanje raste (do određene granice), zatim sa porastom broja procesora efikasnost opada i $S(n)*E(n)$ je najbolja ocena.

2. Koji se hazardi javljaju kod protočnih sistema? x 5

Odg!ppt3 Hazardi su situacije koje sprečavaju da izvršenje instrukcije otpočne u predviđenom ciklusu.

Hazardi redukuju idealne performanse protočnog sistema (izvršenje jedne instrukcije po klok ciklusu).

Hazardi se mogu klasifikovati u tri grupe:

- Strukturni hazardi – nastaju zbog jednovremenih zahteva za korišćenjem istog hardverskog resursa
- Hazardi po podacima – nastupaju zato što je redosled pristupa operandima izmenjen uvođenjem protočnosti
- Kontrolni hazardi – nastupaju zbog zavisnosti u redosledu izvršenja instrukcija (izazivaju ih instrukcije koje mogu promeniti sadržaj PC)

3. Ako su I_j i I_k instrukcije u programu, pri čemu I_i prethodi I_k , objasniti kada nastupaju RAW, WAR i WAW hazardi. x 2

Odg!ppt3 Ako su I i J dve instrukcije, pri čemu I prethodi J , hazardi po podacima se u zavisnosti od redosleda upisa i čitanja mogu klasifikovati u tri grupe:

- RAW (Read After Write) – nastupa kada instrukcija J pokušava da pročita operand pre nego što je instrukcija I obavila upis (najčešći tip hazarda)
- WAR (Write After Read) – nastupa kada instrukcija J pokušava da upiše novu vrednost pre nego što je instrukcija I obavila čitanje
- WAW (Write After Write) – nastupa kada instrukcija J pokušava da upiše vrednost pre instrukcije I
- RAR nije hazard.

WAR i WAW su hazardi imenovanja (name dependencies) a RAW su pravi hazardi (true dependencies)

4. U kojoj fazi protočnog izvršenja instrukcija se detektuju svi hazardi? x 2

Odg!ppt3 Svi hazardi se detektuju u ID fazi (u toku dekodiranja)

5. Šta su kontrolni hazardi? **Odg!ppt3**

Kontrolni hazardi – nastupaju zbog zavisnosti u redosledu izvršenja instrukcija.

Mogu uzrokovati veći gubitak performansi nego hazardi po podacima.

Nastupaju zbog instrukcija koje mogu promeniti sadržaj PC (branch, jump, call, return)

6. Kako se rešava problem prekida kod protočnog izvršenja FP instrukcija.

Odg!ppt3: *Zaustavljanje protočnog sistema se može eliminisati tako što se neće zahtevati da se instrukcije izvršavaju po redosledu pribavljanja. Ovo je moguće zato što ID faza obuhvata: proveru strukturnih hazarda i proveru hazarda po podacima. Ako nema strukturnih hazarda, cilj je da izvršenje instrukcije otpočne čim su dostupni njeni operandi. Na taj način može doći do izvršenja instrukcija van redosleda, što će dovesti do završetka-van-redosleda. ID faza se deli na dva dela: ISSUE(odluka, izdavanje) – dekodiranje instrukcije i proveru strukturnih hazarda, Read operands (čitanje operanada)- čekanje na brisanje data hazarda, zatim čitanje operanada. Sve instrukcije prolaze kroz Issue stepen u redosledu pribavljanja, ali u read operands stepenu mogu biti zaustavljene ili propuštene u izvršenje-van-redosleda.*

7. U kojoj fazi protočnog izvršenja instrukcije se **razrešava grananje? ???????**

Odg?arh: *Ishod uslova grananja i adresa odredišta grananja se određuje u trećoj (EX) fazi, programski brojač PC se puni u četvrtoj fazi adresom odredišta grananja, dok će se u petoj fazi pribaviti instrukcija koja sledi instrukciju grananja.*

8. Objasniti tehniku premošćavanja (pribavljanja unapred).

Odg!arh: Tehnika premošćavanja predstavlja efikasan način da se hazardi podataka izbegnu. Ideja ove tehnike jeste da se izvorišni operandi dostave zavisnim instrukcijama pre nego što ih prethodna instrukcija (instrukcija proizvođač) upiše u odredišni registar. Ova tehnika je moguća jer većina instrukcija proizvodi rezultat u trećoj fazi, te se stoga ne mora čekati da se upiše u odredišni registar već se nakon treće faze može dostaviti sledećoj (zavisnoj) instrukciji.

9. Koje faze u izvršenju instrukcija postoje kod Scoreboard a koje kod Tomasulovog algoritma?
x 5 **Odg!ppt3-NI:**Nemoj zaboraviti IF fazu. **Odg!ppt4-NI:**Nemoj zaboraviti IF fazu.

Scoreboard - Četiri koraka koji zamenjuju ID, Ex i WB su (ne razmatramo stepen za obraćanje memoriji):

1. IF faza – pribavljanje instrukcije

2. Issue – ako je funkcionalna jedinica slobodna i ni jedna druga aktivna instrukcija nema isti odredišni registar, Sc. propušta instrukciju u funkcionalnu jedinicu i ažurira internu strukturu podataka. **Ovde se detektuju strukturni i WAW hazardi.** Ako strukturni ili WAW hazard postoji, onda se izdavanje svih instrukcija zaustavlja dok se hazard ne obriše. Kada se Issue stepen zaustavi, bafer između IF i Issue stepena će se napuniti i prestaće da se pribavljaju nove instrukcije. Ključno je uočiti da instrukcije kroz Issue stepen prolaze po redosledu pribavljanja.

3. Read operands – Sc. nadgleda raspoloživost izvornih operanada. Izvorni operand je raspoloživ ako ni jedna druga aktivna instrukcija koja je prethodno izdata neće da ga generiše (upiše rezultat). Kada su operandi dostupni, Sc. kaže funkcionalnoj jedinici da pročita operande iz RF i otpočne izvršenje. Sc. rešava RAW hazarde dinamički u ovom koraku i instrukcija može biti poslata u izvršenje van redosleda.

4. Execution (izvršenje) – Funkcionalna jedinica (FU) otpočinje izvršenje nakon pribavljanja operanada. Kada je rezultat spreman FU obaveštava Sc.

5. MEM faza - Obraćanje memoriji /okončanje grananja

6. Write results (upis rezultata) - kada je Sc. obavešten da je FU okončala saizvršenjem, proverava WAR hazarde i zaustavlja instrukciju ako je potrebno. Ako WAR ne postoji, ili se obriše, SC. kaže FU da upiše rezultat u odredišni registar.

Tomasulo:

1. IF faza – pribavljanje instrukcije
2. Issue – instrukcija se pribavlja iz reda čekanja. Ako je u pitanju FP operacija, instrukcija se izdaje ako postoji slobodna rezervaciona stanica i šalju se operandi ako su u registrima. Ako je u pitanju load ili store instrukcija, ona se izdaje ako postoji slobodan load ili store bafer. Ako nema slobodne rezervacione stanice ili load/store bafera onda postoji strukturni hazard i instrukcija se zaustavlja. U ovom koraku se vrši i proces preimenovanja registara.
3. Execution (izvršenje) – Ako neki operand nije dostupan, nadgleda se CDB. Kada operand postane dostupan smešta se u odgovarajuću RS. Kada su oba operanda dostupna, izvršava se FP operacija. Vršiti se proveru RAW hazarda (čekanjem da operandi postanu dostupni razrešavaju se RAW hazardi). Load i store zahtevaju dvostepeno izvršenje: u prvom koraku se izračunava efektivna adresa, a zatim se pamti u load/store bafer, Pribavljanje u load bafer se obavlja čim je memorijska jedinica raspoloživa, upisi u store bafere čekaju na podatak pre nego što se obavi upis u memoriju.
4. MEM faza - Obraćanje memoriji /okončanje grananja
5. Write result (upis rezultata) – kada je rezultat dostupan upisuje se na CDB, a odatle u registre i bilo koju FU koja čeka taj rezultat. Označiti da je FU slobodna.

10. U kom stepenu protočnog sistema koji koristi Scoreboard dolazi do preuređenja instrukcija koje dovodi do "izvršenja-van-redosleda"?

Odg!ppt3: Read faza.

11. Koje aktivnosti se obavljaju u ISSUE fazi kod x2

- a. Scoreboard tehnike **Odg!ppt3** u 9. pitanju
- b. Tomasulovog algoritma **Odg!ppt4**

12. Pobrajati razlike između Scoreboard tehnike i Tomasulovog algoritma. x4 **Odg!ppt4**

Upravljanje – Distribuirano kod TA, centralizovano kod SC; preimenovanje registara (citanje vrednosti) kod TA, kod SC se koriste registri opšte namene; Rezultati – kod TA se prosledjuju magistralu (CDB), a kod SC se upisuju u registre; LOAD i STORE se tretiraju kao FU

13. U kojoj fazi izvršenja instrukcija se pristupa Baferima predikcije grananja (BPB) a u kojoj Baferima ciljne adrese grananja (BTB)?

Odg!ppt5: Za BPB u ID fazi, a za BTB u IF.

BPB je mala memorija indexirana nizim bitovima adrese naredbe grananja. Najjednostavnija koristi samo 1 bit (1 ako se grananje nedavno obavilo, 0 ako nije). Koristi se kada je adresa grananja poznata pre uslova grananja. Pristupa joj se ID fazi. Problem – predviđanje je netacno u prvoj i poslednjoj fazi unutrašnje petlje. To se ispravlja 2-bitnom semom. Ako je predviđanje 2 puta netacno menja se predikcija.

14. Opisati rad n-bitnog prediktora za predikciju grananja. x2

Odg!ppt5 N-bitne šeme koriste n-bitni zasićeni brojač za predikciju grananja.

Vrednosti koje brojač može uzeti su u opsegu $0 - 2^n - 1$.

Ako je vrednost brojača $\Rightarrow 2^{(n-1)}$ predviđanje je da će se grananje obaviti (taken), u suprotnom da se ne obavlja (not taken).

Brojač se inkrementira svaki put kada se grananje obavi, a dekrementira kada se grananje ne obavi. Kada je vrednost brojača $2^n - 1$, nema inkrementiranja.

Kada je vrednost brojača 0, nema dekrementiranja.

Proučavanja n-bitnih prediktora su pokazala da se 2-bitni prediktori ponašaju skoro isto dobro kao n-bitni ($n > 2$) prediktori. Zbog toga većina sistema koristi 2-bitne prediktore.

15. Odrediti srednje gubitke u clk za naredbu grananja ako se koristi BTB usvajajući da je: pouzdanost predviđanja 80%, stopa pogotka u BTB 90%, grananja se dešavaju u 70% slučajeva. Broj izgubljenih clk ciklusa u slučaju pogrešne predikcije i u slučaju da se naredba ne nađe u BTB iznosi 2 dk ciklusa.

Odg!ppt5: $gubitci = stopa_pogotka * pogrešna_predikcija * broj_clk_za_pogrešnu_predikciju + (1 - stopa_pogotka) * grananje_se_dešava * broj_clk_naredba_nije_u_BTB$
 $gubitci = 0,9 * 0,2 * 2 + 0,1 * 0,7 * 2 = 0,5clk$

16. Pobrojati hardverske tehnike za smanjenje kašnjenja uzrokovanog kontrolnim hazardima. x 2

Odg!ppt5: *To su zapravo tehnike za dinamičku predikciju grananja: jedno-nivovske (BPB), korelacioni prediktori, baferi ciljne adrese grananja (BTB) i hibridni prediktori.*

17. Koje su osnovne komponente vektorskog računara? x 5

Odg!ppt6: *Memorija, skalarni podsistem, vektorski podsistem.*

18. Ako postoje ugnježđene petlje, po kojoj petlji se vrši vektorizacija? x 4

Odg!ppt7: *Najdublje ugnježđene petlje.*

19. Koje se elementarne transformacije mogu obavljati nad indeksnim promenljivim u petljama? x 3

Odg!ppt7: *permutacije, obrtanje redosleda i krivljenje.*

20. Kako se određuju vektori zavisnosti po podacima? x 4 **Odg!ppt7**

Ako je generisana promenljiva $X(f(i))$, gde je $f(i)$ celobrojna f-ja definisana nad indexnim skupom i , a $X(g(i))$ korisćena promenljiva, vektor zavisnosti se određuje kao: $d(i) = f(i) - g(i)$

21. Koje vrste zavisnosti sprečavaju vektorizaciju? Koje osobine moraju da zadovolje matrice transformacije indeksnih promenljivih? x 6

Odg!ppt7: *Raw hazardi (Kada naredna iteracija koristi rezultat koji generise predhodna iteracija). Tri osobine: kvadratna matrica (što znači da se vrši preslikavanje n-dimenzionalnog indeksnog prostora u n-dimenzionalni indeksni prostor), celobrojna matrica, $\det = 1$.*

22. Data je mat. zavisnosti po podacima $D = [d_1 \ d_2 \ d_3] = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 2 \end{bmatrix}$ i mat. transf. $T = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$.

Da li je dozvoljeno izvršiti transformaciju petlji primenom transformacije T? Zašto?

Odg?ppt7: *Da bi jedna transformacija mogla da se primeni nad indeksnim skupom, a da to ne utiče na korektnost izračunavanja, matrica transformacije T ne sme da menja znak vektora zavisnosti. d_1, d_2, d_3 su pozitivni. Sada računamo vektore d kapa, pa tako je $d_1kapa =$*

$$T * d_1 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}. \text{ Iz ovoga sledi da je } d_1kapa \text{ negativna, što znači da matrica}$$

transformacije menja znak vektora zavisnosti pa nije dozvoljeno izvršiti transformaciju petlji.

23.

- a. Naći sve vektore zavisnosti po podacima u sledećim gnezdu petlji

```
for i:=1 to n
for j:=1 to n
for k:=1 to n
a(i,j,k):= a(i-1,j+1,k)+a(i,j,k-1)+b(i,j-1,k)
b(i,j,k):=b(i,j-1,k+1)+a(i-1,j,k)
```

Odg?ppt7: Parovi generisanih-korišćenih promenljivih:

$A(i,j,k)$ i $A(i-1,j+1,k)$

$A(i,j,k)$ i $A(i,j,k-1)$

$A(i,j,k)$ i $A(i-1,j,k)$

$B(i,j,k)$ i $B(i,j-1,k)$

$B(i,j,k)$ i $B(i,j-1,k+1)$

odgovarajući vektori zavisnosti:

$$d_1 = (i,j,k)^T - (i-1,j+1,k)^T = [1 \ -1 \ 0]^T$$

$$d_2 = (i,j,k)^T - (i,j,k-1)^T = [0 \ 0 \ 1]^T$$

$$d_3 = (i,j,k)^T - (i-1,j,k)^T = [1 \ 0 \ 0]^T$$

$$d_4 = (i,j,k)^T - (i,j-1,k)^T = [0 \ 1 \ 0]^T$$

$$d_5 = (i,j,k)^T - (i,j-1,k+1)^T = [0 \ 1 \ -1]^T$$

- b. Da li je dozvoljeno izvršiti permutaciju petlji po indeksnim promenljivima j i k?

Odg?ppt7: Da bi jedna transformacija mogla da se primeni nad indeksnim skupom, a da to ne utiče na korektnost izračunavanja, matrica transformacije T ne sme da menja znak vektora zavisnosti. d_1, d_2, d_3, d_4, d_5 su pozitivni. Sada računamo vektore d kapa,

$$\text{pa tako je } d_1\text{kapa} = T * d_1 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}. \text{ Iz ovoga sledi da je } d_1\text{kapa}$$

negativa, što znači da matrica transformacije menja znak vektora zavisnosti pa nije dozvoljeno izvršiti permutaciju.

24. Ako je latentnost memorije 8 clk ciklusa a vektorski korak 10 koliko je najmanje mem. banaka potrebno da bi se elementima vektora pristupilo bez konflikata ako broj banaka mora da bude stepen dvojke.

Odg?ppt7: Da bi se elementima vektora pristupilo bez konflikata treba da vazi:

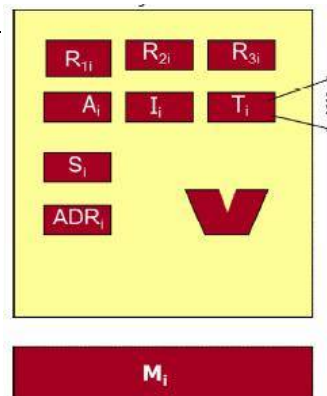
$NZS(VK,X)/VK > Lat$ (VK – vektorski korak, X – broj memorijskih banaka, Lat – Latentnost).

$$X=32$$

25. Navesti vrste maskiranja procesnih elemenata kod SIMD sistema. x 2

Odg!ppt9: direktno (maskiranje proizvoljnih podskupova PE),

maskiranje pomoću adrese PE i dinamičko (maskiranje podacima)



26. Nacrtati strukturu procesnog elementa kod procesorskih polja

sa distribuiranom memorijom. x 2 **Odg!ppt9**

R_{1i} , R_{2i} , R_{3i} – radni registri, A_i – adresni registar, I_i – indeksni registar (za adresiranje lokalne memorije zajedno sa A_i), T_i – registar za komunikaciju spregnut sa drugim T regis. preko S Mreže, S_i – statusni registar (0 Pe pasivan/1 PE aktivan), ADR_i – m-bitna adresa PE-a.

27. U SIMD sistemu postoji 16 procesnih elemenata (PE). PE-ovima sa adresama 0, 8, 9, 12, 13 i 15 treba dozvoliti rad a ostalima zabraniti. Kako izgledaju maske kojima se ovo postiže ako se koristi direktno maskiranje a kako ako se koristi maskiranje adrese?

Odg!ppt9: direktno -> 1011001100000001 (vektor maske, pri čemu svaki bit odgovara jednom PE), maskiranje adrese -> adresa PE0 je 0000, PE1-0001... nije moguće maskirati pomoću jedne maske, već su nam potrebne tri maske: 0000, 1111 i 1x0x

28. Kojim sprežnim funkcijama su definisane sledeće sprežne mreže: x 7

a. Mešanje-zamena (shuffle-exchange),

Odg!ppt10: SF-mešanja -> $S(b_{m-1}, b_{m-2}, \dots, b_1, b_0) = b_{m-2}, \dots, b_1, b_0, b_{m-1}$

SF-zamena -> $E(b_{m-1}, b_{m-2}, \dots, b_1, b_0) = b_{m-1}, b_{m-2}, \dots, b_1, \bar{b}_0$

b. kub,

Odg!ppt10: $C_i(b_{m-1}, \dots, b_i, \dots, b_1, b_0) = b_{m-1}, \dots, \bar{b}_i, \dots, b_1, b_0$, $m = \log_2 N$
 $i = 0, 1, \dots, m-1$

c. rešetka (mesh)

Odg!ppt10: Definisana sa 4 SF: $M_{+1}(x) = (x+1) \bmod N$, $M_{-1}(x) = (x-1) \bmod N$,
 $M_{+r}(x) = (x+r) \bmod N$, $M_{-r}(x) = (x-r) \bmod N$, $r = \sqrt{N}$

29. Kako se određuje zaglavlje poruke koje definiše put kroz sprežnu mrežu generalizovani kub u slučaju da se vrši emisija? x 2

Odg!ppt10: Koristi se proširena sema sa EX-OR zaglavljem. Broj odredista mora biti stepen dvojke (2 na i). Odredisne adrese se moraju razlikovati na j pozicija, a preklapati na m-j pozicija. Zaglavlje za slučaj emisije određeno je skupom $\{R, B\}$, $R = r_{m-1} \dots r_1 r_0$ sadrži informaciju o putu, $B = b_{m-1} \dots b_1 b_0$ informacije o tačkama grananja. Imamo $S=101$, $E=100$, $F=110$. $Te=001$ (0 ako su bitovi jednaki, 1 ako nisu), $Tf=011$. Te i Tf se razlikuju samo na srednjem bitu. Za R se uzima bilo koji (npr. Te), a $B = (TE + Tf) \bmod 2 = 010$ Ako je $bi=1$, ri se ignorise i obavlja se emisija, ako je $bi=0$, ri se koristi da se odredi položaj KE

30. U sistemu postoji 16 procesora. Za povezivanje se koristi višestepena sprežna mreža generalizovani kub. Procesor0 treba da pošalje poruku procesorima 4,5,6,7,12,13,14 i 15.

a. Da li je emisija moguća? Zašto?

Odg!ppt10: Da bi emisija bila moguća broj odredišta mora biti stepen dvojke. U ovom slučaju broj odredišta je 8, $2^3=8$. Takođe treba proveriti i da li se te adrese poklapaju na tačno 4-3 pozicija (4 zbog 2^4 procesora). U ovom slučaju jeste tačno jer se od 4 bita svuda poklapa 3. sa desna, pa može da se radi emisija.

b. Kako izgleda zaglavlje koje se dodaje poruci?

Odg!ppt10: S -> izvor $S=0000$, D -> odreditš $D_4=0100$, $D_5=0101 \dots D_{15}=1111$
 T_i -> individualno zaglavlje $T_4=S \oplus D_4=0100$, $T_5=S \oplus D_5=0101 \dots T_{15}=S \oplus D_{15}=1111$
 R -> informacija o putu, uzima se bilo koji T_i , npr. T_4
 B -> informacija o tačkama grananja, $B=T_4 \oplus T_5 \oplus T_6 \oplus \dots \oplus T_{15}=0100$
Zaglavlje izgleda: $\{R, B\} = \{0100, 0100\}$

31. Navesti razlike između multiprocesora i multiračunara. **Odg!ppt11**

MultiProcesori – Cvrsto spregnuti, zajednička memorija i skup U/I uređaja, komunikacija preko deljive memorije, upravlja jedinstveni OS.

MultiRacunari – Slabo spregnuti, heterogeni, sopstvene memorije i U/I uređaji, komunikacija preko sprezne mreže putem poruka.

32. Navesti politike dodele magistrale kod multiprocesorskih sistema. x 6

Odg!ppt11: Politike mogu biti zasnovane na prioritetu, nepristrasnosti i kombinovana.

33. Opisati centralizovanu arbitražu koja se zasniva na zajedničkim zahtevima i lančanom zahvatanju. x 2 **Odg!ppt11**

Deljivi zahtevi: Svaki potencijalni gospodar salje zahtev preko BUS REQUEST linije. Centralni arbitar salje BUS GRANT signal gospodaru sa brojem 1. BUS GRANT povezuje sve gospodare u lanac (daisy chain), tako da gospodar broj 1 salje signal broju 2... Gospodar koji je poslao zahtev aktivira liniju BUS BUSY, a kad završi oslobadja je. Politika fiksnih prioriteta (blizi arbitru -> veci prioritet). Jednostavna sa malim brojem linija.

34. Objasniti distribuiranu arbitražu na magistrali. x 2

Odg!ppt11 Svaki potencijalni gospodar ima sopstveni arbitar i jedinstveni arbitracioni broj koji se koristi da razreši sudare kod jednovremenih zahteva za korišćenjem magistrale. Kada dva ili više uređaja konkurišu za korišćenje magistrale, pobednik je onaj koji ima viši arbitracioni broj. Svaki potencijalni gospodar može poslati svoj arbitracioni broj (AB) na deljive request/grant linije preko svog arbitra. Od svih zahteva se formira zbirni arbitracioni broj, #AB. Nakon ovoga, svaki arbitar poredi svoj AB sa zbirnim, počev od bita najveće težine. Ako je njegov broj niži od zbirnog to znači da je njegov prioritet niži i zahtev se uklanja. Na kraju na linijama ostaje arbitracioni broj onog gospodara koji ima najviši prioritet i njemu se dodeljuje magistrala.

35. Navesti tehnike za postizanje keš koherencije. x 2

Odg!ppt11: HW tehnike: snoop keš protokoli (Write-once – koristi invalidaciju eng. write-invalidate i firefly – koristi ažuriranje pri upisu eng. write-update) i direktorijumske šeme (protokoli sa potpuno preslikanim adresarima, protokoli sa ograničenim adresarima i protokoli sa ulančanim adresarima). SW tehnike se mogu primeniti u višim programskim jezicima i to na način da se podaci označe kao deljivi ili nedeljivi.

36. Objasniti aktivnosti i promene stanja keš kopija/bloka u slučaju Write-once protokola kada nastupi promašaj pri upisu. x 5

Odg!ppt11: Kada nastupi promašaj pri upisu procesor mora prvo pribaviti kopiju iz glavne memorije ili iz keša koji ima dirty kopiju. Ovo se postiže slanjem Read_Inv komande koja će pribaviti i ažurirati blok i invalidirati sve ostale keš kopije. Lokalna kopija nakon ovoga se nalazi u stanju dirty.

37. Koja su moguća stanja keš bloka kod Write-once protokola (snoopy sa invalidacijom pri upisu)?

Odg!ppt11: Protokol razlikuje četiri stanja kopije keš bloka:

1. Valid (važeća): U kešu je kopija koja je konzistentna (jednaka) sa memorijskom kopijom i nije modifikovana (tj. konzistentna je sa ostalim kopijama, ako postoje, u drugim keševima).
2. Invalid: nevažeća kopija (tj. kopija je poništena)
3. Reserved (rezervisano): podatak je u kešu modifikovan samo jednom i keš kopija je konzistentna sa kopijom u glavnoj memoriji, koja je jedina druga (sve ostale kopije, ako su postojale su invalidirane)
4. Dirty (prljava): keš je modifikovan više puta i kopija je jedinstvena u sistemu. U sistemu može postojati samo jedna prljava kopija.

38. Koja stanja keš kopije/bloka postoje u slučaju Firefly protokola? x 6

Odg!ppt11: Postoje tri moguća stanja keš kopije:

1. *Valid-exclusive* - postoji samo jedna keš kopija i ona je konzistentna sa glavnom memorijom.
2. *Shared* - postoji više kopija i sve su konzistentne
3. *Dirty* - postoji samo jedna kopija i ona nije konzistentna sa glavnom memorijom.

39. Ako nastupi promašaj pri upisu kod obraćanja lokalnom kešu koje se akcije preduzimaju i koje je rezultujuće stanje lokalnih i udaljenih kopija ako se koristi:

a. Write-once protokol

Odg!ppt11: kada nastupi promašaj pri upisu procesor mora prvo pribaviti kopiju iz glavne memorije ili iz keša koji ima dirty kopiju. Ovo se postiže slanjem *Read_Inv* komande koja će pribaviti i ažurirati blok i invalidirati sve ostale keš kopije. Lokalna kopija nakon ovoga se nalazi u stanju dirty.

b. Firefly protokol

Odg!ppt11: kopija dolazi iz drugih keševa ili iz glavne memorije. ako dolazi iz memorije stanje kopije nakon upisa je dirty. U ostalim slučajevima sve kopije se ažuriraju a novo stanje je shared.

40. Multiprocesorski sistem se sastoji od tri procesora. Svaki procesor ima svoju keš memoriju. Prikazati kako se vrši promena stanja keš blokova i glavne memorije ako se za održavanje keš koherencije koristi:

a. Write-once protokol

b. Firefly protokol

Procesori pristupaju deljivoj promenljivoj X. Akcije procesora navedene su u prvoj koloni tabele. Popuniti ostale ćelije tabele. x 2

Akcija procesora	Stanje keš kopije u P1		Stanje keš kopije u P2		Stanje keš kopije u P3		Podatak se pribavlja iz GM ili keša		Stanje kopije u glavnoj memoriji	
	a)	b)	a)	b)	a)	b)	a)	b)	a)	b)
P1 čita X	v	v-e	/	/	/	/	gm	Gm	važeća	važeća
P3 čita X	v	s	/	/	v	S	gm	Gm	važeća	važeća
P3 upisuje u X	i	s	/	/	r	S	keš	keš	važeća	važeća
P3 upisuje u X	i	s	/	/	d	S	keš	keš	nevažeća	važeća
(21.1.2009) P2 čita X	i	s	v	s	v	S	keš	Gm	važeća	važeća
(21.1.2011) P2 upisuje u X	i	s	d	s	i	S	keš	keš	nevažeća	važeća

41. Šta su direktorijumske šeme? x 3 **Odg!ppt11**

Kada se koriste vešestepene sprežne mreže za gradnju multiprocesorskih sistema sa velikim brojem procesora, snoopy protokoli se moraju modifikovati da bi se prilagodili mogućnostima mreže. Postoji broadcast (emisiju) veoma skupo obaviti kod višestepenih mreža, komande konzistentnosti se šalju samo onim keševima koji imaju kopiju bloka. Da bi se to učinilo potrebne su tačne informacije o tome koje keš memorije imaju kopiju određenog bloka. Ove informacije se smeštaju u direktorijumima (adresarima).

Protokoli koji koriste informacije zapamćene u direktorijumima za održavanje koherentnosti memorijskog sistema zovu se direktorijumske šeme. Svaki memorijski modul sadrži poseban direktorijum koji beleži stanje i prisustvo memorijskog bloka u određenom kešu. Svaki direktorijumski ulaz (linija) sadrži veći broj pokazivača koji ukazuju na keš koji sadrži kopiju

bloka i jedan "dirty" bit kojim se ukazuje kada određeni keš ima, ili ne, dozvolu da modikuje podatak. Postoje tri tipa direktorijumskih protokola: protokoli sa potpuno preslikanim adresarima (Full-map directories), protokoli sa ograničenim adresarima (limited directories), protokoli sa ulančanim adresarima (chained directories).

42. Kako se održava konzistentnost podataka kod direktorijumskih šema? x 2

Odg!ppt11 Komande konzistentnosti se šalju samo onim keševima koji imaju kopiju bloka. Da bi se to učinilo potrebne su tačne informacije o tome koje keš memorije imaju kopiju određenog bloka. Ove informacije se smeštaju u direktorijumima (adresarima).

43. Objasniti kako izgleda formiranje direktorijumskog lanca ako se koristi direktorijumska šema sa ulančanim adresarima.

Odg!ppt11 Ako P1 čita lokaciju X, glavna memorija šalje kopiju bloka kešu procesora P1 i pokazivač na kraj lanca CT (chain termination). U adresaru glavne memorije čuva pokazivač na procesor P1. Ako P2 zahteva čitanje X, glavna memorija šalje kopiju bloka kešu procesora P2 i pokazivač na P1, a u adresaru glavne memorije čuva pokazivač na P2. Ponavljanje koraka, svi keševi mogu dobiti kopiju bloka.

44. Koje su operacije dozvoljene nad semaforima? Dati opis operacija u pseudo kodu. x 2

Odg!ppt12: *Dozvoljene su operacije P i V (Wait i Signal).*

P(S): if $S > 0$ then $S := S - 1$

else { blokirati proces koji je izvršio P(S) i smestiti ga u red čekanja pridružen semaforu S }

V(S): $S := S + 1$ if { red čkanja pridružen semaforu S nije prazan }

then { selektovati jedan proces iz reda čekanja i smestiti ga u red spremnih }

45. Čemu služe podaci tipa CONDITION kod monitora i koje se operacije mogu izvršavati nad podacima tog tipa? x 3

Odg!ppt12: *Da bi se obezbedila potrebna sinhronizacija procesa unutar monitora se deklariše i poseban tip podatka CONDITION. Ovi podaci se koriste kada proces želi da zakasni svoje izvršenje ili pobudi prethodno blokirani proces. Na ovaj način je deklarisan red čekanja usl. Nad promenljivima tipa CONDITION moguće je izvršavati samo dve opracije: WAIT(usl) i SIGNAL(usl). WAIT(usl) operacija blokira proces i dodaje ga redu čekanja pridruženom promenljivoj usl. Ova operacija automatski oslobadja pristup monitoru, tako da drugi procesi mogu da udju u monitor. SIGNAL(usl) omogućava da se aktivira proces koji je bio blokirani zbog usl. Ako nema ni jednog procesa koji čeka, ova naredba je bez dejstva.*

46. Navesti razlike između semafora i monitora kao sredstva sinhronizacije i komunikacije procesora. x 2 **Odg!ppt12**